

程序设计实践

The Practice of Programming

记法(Notation)

广义的“记法(Notation)”

Scientific Notation

玻尔半径 = $5.2917721067 \times 10^{11}$ 米

Musical Notation



JavaScript Object Notation (JSON)

```
{ "book": [ { "id": "01", "language": "Java", "edition": "third", "author": "Herbert Schildt" }, { "id": "07", "language": "C++", "edition": "second", "author": "E.Balagurusamy" } ] }
```

Abstract Syntax Notation 1 (ASN.1)

```
MessageType ::= CHOICE {  
    unidirectional  
    begin  
    end  
    continue  
    abort  
    [APPLICATION 1] IMPLICIT Unidirectional,  
    [APPLICATION 2] IMPLICIT Begin,  
    [APPLICATION 4] IMPLICIT End,  
    [APPLICATION 5] IMPLICIT Continue,  
    [APPLICATION 7] IMPLICIT Abort }
```

本章所谓“记法”的定义

基于一个可打印字符集合，用集合中的字符的特定组合来**形式化**地表达特定的数据或者逻辑的方法。

表达逻辑：

各种能够表达逻辑的领域特定语言，各种程序设计语言...

表达数据结构：

xml, json, asn.1, protobuf, LaTeX, HTML...

表达数据元素，动作：

整数，浮点数，字符，字符串，名字，格式控制，命令，操作符...

简单的记法

复杂的记法

记法是人与程序接口的重要组成部分

人

- ✓ 使用可显示的文字符号，便于人阅读
- ✓ 使用可编辑的文字符号，便于人编辑
- ✓ 使用具有特定含义的文字符号，便于人理解

好的记法

程序

- ✓ 严格的词法语法，便于分析
- ✓ 简洁的表达方式，提高处理效率
- ✓ 明确的含义，便于程序解释

记法是人与程序接口的重要组成部分

人

字符串 Hello, world! 加上回车

"Hello, world!\n"

程序
(编译器)

48	65	6C	6C	6F	2C	20	77	6F	72	6C	64	21	0A	00
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

记法是人与程序接口的重要组成部分

人

Unicode序号为128512的符号 (笑脸)

😀

程序
(浏览器)



所有的记法（或语言）都不是天生的，都是程序员在实践中为了方便使用创造出来的。

好的记法，有的一直在小范围内使用，有的则变成了标准，为全球程序员所广泛使用。

如果觉得现有的记法都不能很好地贴合你的需求，何不尝试创造一个新记法？

数据格式化记法

```
printf("%d %6.2f %-10.10s\n", i, f, s);
```

将字符串s的前10个字符输出，最大列宽为10，左对齐

将浮点数f输出，整数部分最大列宽为6，保留2位小数

将整数i按照十进制格式输出

```
time_t t;  
time(&t);  
struct tm* ptm = localtime(&t);  
char time_str[256];  
strftime(time_str, sizeof(time_str), "%Y-%m-%d %T", ptm);
```

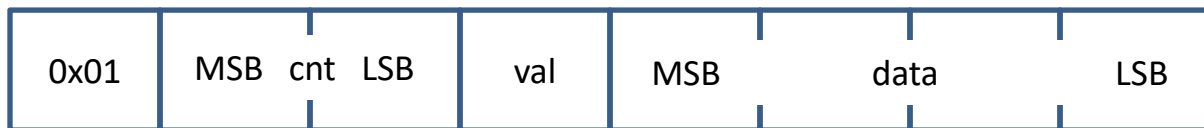
这里输出当前时间的4位年份

这里输出当前时间的2位月份

这里输出当前时间日期部分，2位数字表示

当前时间的时分秒信息，按照HH:MM:SS格式输出

设计一个数据格式化记法 - 需求



- cnt是2字节的整数，高位低地址。
- data, data2都是4字节的整数，高位低地址。



A和B之间传输的应用协议数据单元

TCP/IP



如何打包这些协议数据单元？

设计一个数据格式化记法 - 普通实现

```
// 针对4字节整数打包的通用函数，返回后p会被改变
void pack_int(unsigned char*& p, unsigned int data){
    *p++ = data >> 24; //将左数第1个字节赋值给*p
    *p++ = data >> 16; //将左数第2个字节赋值给*p
    *p++ = data >> 8;  //将左数第3个字节赋值给*p
    *p++ = data;       //将左数第4个字节赋值给*p
}
```

```
// 0x01消息的打包函数，返回打包长度
int pack1(unsigned char* buf
    , unsigned short cnt
    , unsigned char val
    , unsigned int data){
    unsigned char* p = buf;
    *p++ = 0x01;
    *p++ = cnt >> 8; //保存高位字节
    *p++ = cnt; //保存低位字节
    *p++ = val;
    pack_int(p, data);
    return p - buf;
}
```

```
// 0x02消息的打包函数，返回打包长度
int pack2(unsigned char* buf
    , unsigned short cnt
    , unsigned int data
    , unsigned int data2) {
    unsigned char* p = buf;
    *p++ = 0x02;
    *p++ = cnt >> 8;
    *p++ = cnt;
    pack_int(p, data);
    pack_int(p, data2);
    return p - buf;
}
```

设计一个数据格式化记法 - 格式化记法实现

```
// fmt是一个由c,s,l组成的字符串, c表示char, s表示2字节整数, l表示4字节整数
int pack(unsigned char* buf, const char* fmt, ...)
    unsigned char* p = buf;
    循环依次读取fmt的每一个字节
        如果是c, 则从后续参数列表中取下一个参数v, *p++ = v;
        如果是s, 则从后续参数列表中取下一个参数v, *p++ = v << 8; *p++ = v;
        如果是l, 则从后续参数列表中取下一个参数v, pack_int(p, v);
    return p - buf;
}
```

```
int pack1(unsigned char* buf, unsigned short cnt, unsigned char val
, unsigned int data){
    return pack(buf, "cscl", 0x01, cnt, val, data);
}
```

```
int pack2(unsigned char* buf, unsigned short cnt, unsigned int data
, unsigned int data2){
    return pack(buf, "csll", 0x02, cnt, data, data2);
}
```

更复杂一些的记法示例

```
grep 'class.*MyClass' *.java
```

grep命令使用正则表达式从所有java程序中找出MyClass类的定义

```
/^[a-fA-F]{6}$/ .test(id);
```

JavaScript中使用正则表达式判断id是否是一个6位的十六进制数字

```
cat b.txt | awk '{ for(i = 1; i <= NF; i++) print $i}' > b2.txt
```

使用awk命令将b.txt转换为每个单词一行，输出到b2.txt

```
$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$
```

在Markdown中使用LaTeX记法输出一个公式: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

```
<path d="M250 150 L150 350 L350 350 Z" />
```

SVG中对一个绘图路径的描述

作为程序员，在使用这些他人实现的记法的时候，不应该仅仅使用它们，应该下意识地想想，这个记法是怎么被解释执行的呢？

记法的解释执行 - 最简单的csl记法的例子

```
int pack(unsigned char* buf, const char* fmt, ...){
    va_list args;
    va_start(args, fmt);
    unsigned char* bp = buf;
    const char* fp;
    for(fp = fmt; *fp != '\0'; fp++){
        switch(*fp){
            case 'c':
                *bp++ = va_arg(args, int);
                break;
            case 's':
                pack_short(bp, va_arg(args, int));
                break;
            case 'l':
                pack_int(bp, va_arg(args, unsigned int));
                break;
            default:
                va_end(args);
                return -1;
        }
    }
    va_end(args);
    return bp - buf;
}
```

循环对fmt中的记法字符串
的每一个字符进行处理

如果遇到c怎么处理...

如果遇到s怎么处理...

如果遇到l怎么处理...

其它字符都是语法错误，
错误处理。

Anchors

<code>^</code>	Start of string, or start of line in multi-line pattern
<code>\A</code>	Start of string
<code>\$</code>	End of string, or end of line in multi-line pattern
<code>\Z</code>	End of string
<code>\b</code>	Word boundary
<code>\B</code>	Not word boundary
<code>\<</code>	Start of word
<code>\></code>	End of word

Groups and Ranges

<code>.</code>	Any character except new line (<code>\n</code>)
<code>(a b)</code>	a or b
<code>(...)</code>	Group
<code>(?:...)</code>	Passive (non-capturing) group
<code>[abc]</code>	Range (a or b or c)
<code>[^abc]</code>	Not (a or b or c)
<code>[a-q]</code>	Lower case letter from a to q
<code>[A-Q]</code>	Upper case letter from A to Q
<code>[0-7]</code>	Digit from 0 to 7
<code>\x</code>	Group/subpattern number "x"

Quantifiers

<code>*</code>	0 or more	<code>{3}</code>	Exactly 3
<code>+</code>	1 or more	<code>{3,}</code>	3 or more
<code>?</code>	0 or 1	<code>{3,5}</code>	3, 4 or 5

Add a `?` to a quantifier to make it ungreedy.

正则表达式

<https://cheatography.com/davechild/cheat-sheets/regular-expressions/pdf/>

记法的解释执行 - 正则表达式 (1)

实现正则表达式的一个子集:

^: 匹配开始

\$: 匹配结束

.: 匹配任意字符

*****: 匹配重复任意次

```
regexp: "abc"
text:   "aabcd"    => 匹配
        "abdd"      => 不匹配
```

```
regexp: "a.c"
text:   "aabcd"    => 匹配
        "aadcd"    => 匹配
        "acbc"      => 不匹配
```

```
regexp: "ab*c"
text:   "aabcd"    => 匹配
        "aacd"      => 匹配
        "aabbbbbcd" => 匹配
        "aaxcd"      => 不匹配
```

```
regexp: "a.*c"
text:   "abcd"      => 匹配
        "acd"       => 匹配
        "axxxcd"    => 匹配
```

```
regexp: "^abc"
text:   "abcd"      => 匹配
        "aabcd"      => 不匹配
```

```
regexp: "abc$"
text:   "aabc"      => 匹配
        "aabcd"      => 不匹配
```

```
regexp: "^abc$"
text:   "abc"       => 匹配
        "aabc"       => 不匹配
        "abcd"       => 不匹配
```

记法的解释执行 - 正则表达式 (2)

```
bool match(const char* regexp, const char* text) {
    if(regexp[0] == '^')
        return matchhere(regexp + 1, text);
    do{
        if(matchhere(regexp, text))
            return true;
    }while(*text++ != '\0');
    return false;
}
```

// 如果regexp第一个字符是^，那么必须从text头匹配

// 否则，针对text依次匹配，只要遇到一个匹配成功就算成功

// 如果上面的循环中一次也没有成功过，那么只能返回匹配失败

```
bool matchhere(const char* regexp, const char* text) { // 关键递归函数
    if(regexp[0] == '\0')
        return true;
    if(regexp[1] == '*')
        return matchstar(regexp[0], regexp + 2, text);
    if(regexp[0] == '$' && regexp[1] == '\0')
        return *text == '\0';
    if(*text != '\0' && (regexp[0] == '.' || regexp[0] == *text))
        return matchhere(regexp + 1, text + 1);
    return false;
}
```

// 如果regexp跑完了，返回成功

遇到*，调用matchstar，三个参数：
- *之前的那个字符
- *之后的regexp
- 当前text

遇到\$并且\$是regexp的最后一个字符，那么，如果text也跑完了，返回成功，否则返回不成功

如果text未结束，regexp遇到.或者regexp和text的当前字符恰好相等，那么各自指针加1，继续递归匹配

```
bool matchstar(int c, const char* regexp, const char* text) {
    do{
        if(matchhere(regexp, text))
            return true;
    }while(*text != '\0' && (*text++ == c || c == '.'));
    return false;
}
```

如果当前*text一直能匹配c（可以重复的那个字符），那么就一直text++往下试（但是regexp不变）直到匹配成功，或者直到text结束或者遇到不匹配c的。

记法的解释执行 - 算术表达式 (1)

表达式 ::= 项 (加减符号 项) *

项 ::= 因子 (乘除符号 因子) *

因子 ::= 数 | ' (' 表达式 ') '

加减符号 ::= '+' | '-'

乘除符号 ::= '*' | '/'

数 ::= ('0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9') *



记法的解释执行 - 算术表达式 (2)

// 计算“表达式”

```
int eval_expression(const char*& p) {  
    int x = eval_term(p);  
    while(*p == '+' || *p == '-') {  
        if(*p++ == '+')  
            x += eval_term(p);  
        else  
            x -= eval_term(p);  
    }  
    return x;  
}
```

表达式至少含有一个项，先计算这个项，结果放到x

如果后续有+或者-操作符，那么表示还有后续的项

如果是+，则x加上下一个项的计算值

否则是-，则x减去下一个项的计算值

表达式含有的所有项都计算完，返回最终结果

// 计算“项”

```
int eval_term(const char*& p) {  
    int x = eval_factor(p);  
    while(*p == '*' || *p == '/') {  
        if(*p++ == '*')  
            x *= eval_factor(p);  
        else {  
            int y = eval_factor(p);  
            if(y == 0)  
                throw "divide by 0";  
            x /= y;  
        }  
    }  
    return x;  
}
```

项至少含有一个因子，先计算这个因子，结果放到x

如果后续有*或者/操作符，那么表示还有后续的因子

如果是*，则x乘以下一个项的计算值

否则是/，则x除以下一个项的计算值。这里需要检查除数是否为0

项含有的所有因子都计算完，返回最终结果

记法的解释执行 - 算术表达式 (3)

// 计算“因子”

```
int eval_factor(const char*& p) {  
    if(*p == '(') {  
        p++;  
        int x = eval_expression(p);  
        if(*p != ')')  
            throw "missing )";  
        p++;  
        return x;  
    }  
    if(!isdigit(*p))  
        throw "missing ( or digit";  
    int x = atoi(p);  
    while(isdigit(*p))  
        p++;  
    return x;  
}
```

因子有两种可能，第一种是括号括起来的一个表达式，处理掉左括弧和右括弧，中间的部分，递归调用表达式的计算函数，并返回表达式的计算结果，即为因子的计算结果。

因子的第二种可能，是一串数字组成的“数”，用atoi取值之后，跳过这些数字。返回这个转换结果。

With the right notation, many problems become easier. And designing and implementing the notation can be a lot of fun.

有了正确记法的辅佐，许多问题的解决将变得容易得多，而设计和实现这些记法本身，也是趣味无穷的。

